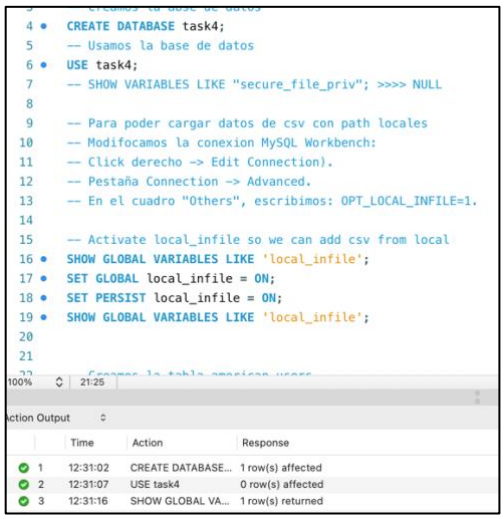
Sprint 4: Table Manipulation

LEVEL 1

- **DATABASE AND TABLES CREATION**

In this case, we insert the corresponding data in each table from local “csv” files.

For that to be possible we need to modify MySQL Workbench permits to instead of loading data from the server directory, to do it from a specified path.



```
CREATE DATABASE task4;
-- Usamos la base de datos
USE task4;
-- SHOW VARIABLES LIKE "secure_file_priv"; >>>> NULL

-- Para poder cargar datos de csv con path locales
-- Modifocamos la conexion MySQL Workbench:
-- Click derecho -> Edit Connection).
-- Pestaña Connection -> Advanced.
-- En el cuadro "Others", escribimos: OPT_LOCAL_INFILE=1.

-- Activate local_infile so we can add csv from local
SHOW GLOBAL VARIABLES LIKE 'local_infile';
SET GLOBAL local_infile = ON;
SET PERSIST local_infile = ON;
SHOW GLOBAL VARIABLES LIKE 'local_infile';
```

100% 21:25

Action Output

	Time	Action	Response
1	12:31:02	CREATE DATABASE...	1 row(s) affected
2	12:31:07	USE task4	0 row(s) affected
3	12:31:16	SHOW GLOBAL VA...	1 row(s) returned

Once this change has been done, we can proceed with tables creation and data loading for each of them. Also, because the size of the data to insert is not known beforehand, the table is created with the maximum size possible for most of the columns (VARCHAR(255)).

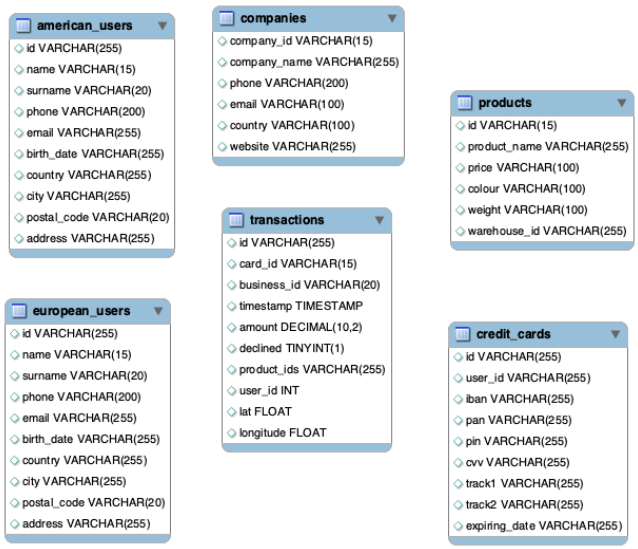


Fig. 1 ER diagram showing tables created

Exercise 1

To get a list of users with more than 80 transactions, we need to do some previous preprocessing and modifications of the tables on the database.

a. We add a new column to the tables "*american_users*" and "*european_users*" called "Continent" with default values for all the users as "Europe" and "America" respectively.

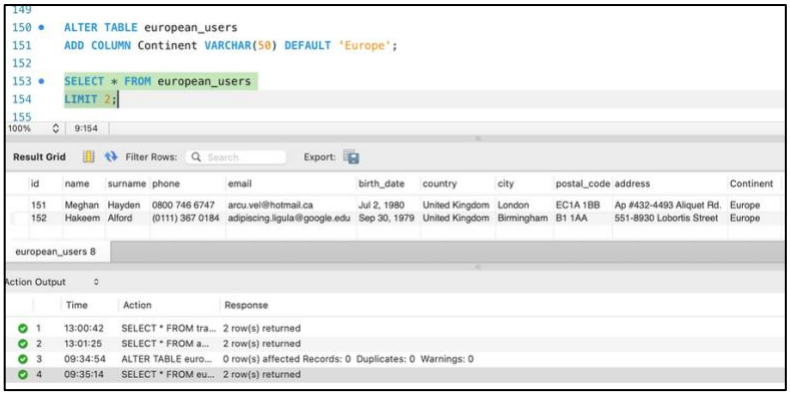


Fig. 2 Modifications on table European users

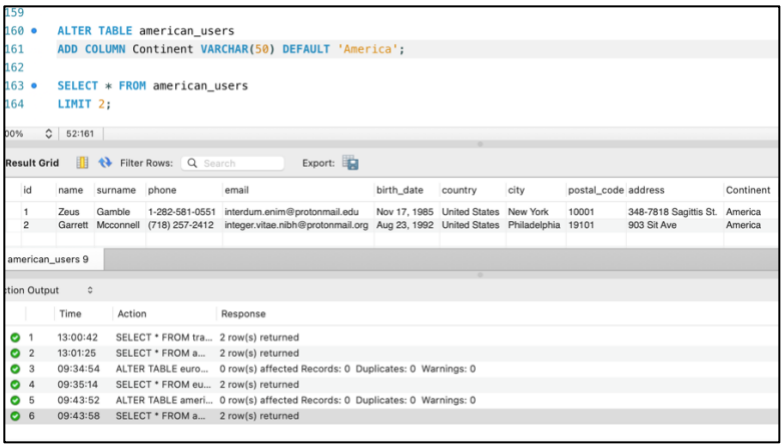


Fig. 3 Modifications on table American users

b. We create a new table "*all_users*", based on the union of tables "*european_users*" and "*american_users*", one under the other one with the columns in the order stablished on the code

c. We modify the tables to set the primary keys and foreign keys. For table "*transactions*" we also modify the datatype of column *user_id* from INT to VARCHAR to set this column as a foreign key.

```sql
-- We set the primary keys in each dimensional and the fact tables:
ALTER TABLE all_users
MODIFY COLUMN id VARCHAR(100) NOT NULL,
ADD CONSTRAINT pk_users_id PRIMARY KEY (id);

ALTER TABLE products
MODIFY COLUMN id VARCHAR(100) NOT NULL,
ADD CONSTRAINT pk_products_id PRIMARY KEY (id);

ALTER TABLE companies
MODIFY COLUMN company_id VARCHAR(100) NOT NULL,
ADD CONSTRAINT pk_company_id PRIMARY KEY (company_id);
```

	Time	Action	Response
1	10:52:22	ALTER TABLE all_u...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
2	10:52:36	ALTER TABLE prod...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
3	10:52:42	ALTER TABLE comp...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0

```sql
ALTER TABLE transactions
    MODIFY COLUMN id VARCHAR(100) NOT NULL,
    MODIFY COLUMN user_id VARCHAR(100) NOT NULL,
    MODIFY COLUMN product_ids VARCHAR(100) NOT NULL,
    MODIFY COLUMN card_id VARCHAR(100) NOT NULL,
ADD CONSTRAINT pk_transactions_id PRIMARY KEY (id),
ADD CONSTRAINT fk_transactions_users_id FOREIGN KEY (user_id) REFERENCES all_users(id),
ADD CONSTRAINT fk_transactions_business_id FOREIGN KEY (business_id) REFERENCES companies(company_id),
ADD CONSTRAINT fk_transactions_credit_cards_id FOREIGN KEY (card_id) REFERENCES credit_cards(id);


DROP TABLES american_users, european_users;
```

	Time	Action	Response
1	12:03:26	ALTER TABLE transactions MODIFY COLUMN id VARCHAR(100) NOT NULL, MODIFY C...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	12:03:39	DROP TABLES american_users, european_users	0 row(s) affected

The table "*credit_cards*" in particular, has a foreign key (user_id) also present in the table "*transactions*". However, we are not going to set this key as foreign in "*credit cards*", table to avoid circular relationships between "*credit_cards*"," *transactions*" and "*all_users*".

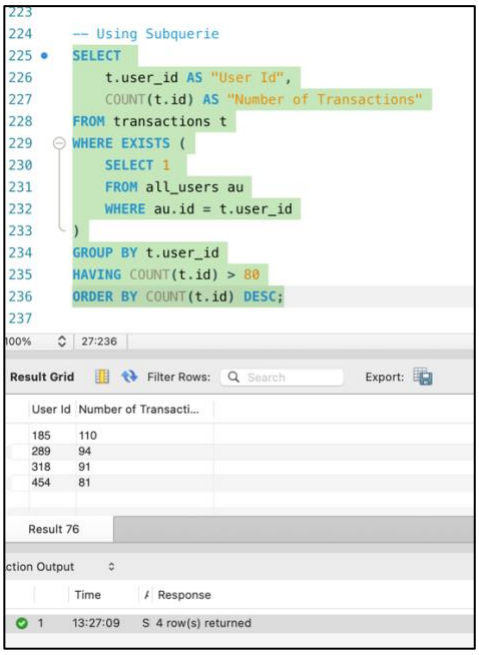
```sql
ALTER TABLE credit_cards
MODIFY COLUMN id VARCHAR(100) NOT NULL,
MODIFY COLUMN user_id VARCHAR(100) NOT NULL,
ADD CONSTRAINT pk_credit_cards_id PRIMARY KEY (id);
-- ADD CONSTRAINT fk_credit_cards_user_id FOREIGN KEY (user_id) REFERENCES all_users(id);
```

	Time	Action	Response
12	11:52:16	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255), card_id VARC...	0 row(s) affected
13	11:52:19	LOAD DATA LOCAL INFILE '/Users/tonzo/Desktop/IT_ACADEMY/sprint4_SQL/transaction...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 W
14	11:52:35	SELECT * FROM transactions LIMIT 2	2 row(s) returned
15	11:52:44	ALTER TABLE european_users ADD COLUMN Continent VARCHAR(50) DEFAULT 'Europe'	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
16	11:52:49	SELECT * FROM european_users LIMIT 2	2 row(s) returned
17	11:52:54	ALTER TABLE american_users ADD COLUMN Continent VARCHAR(50) DEFAULT 'Ameri...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
18	11:52:58	SELECT * FROM american_users LIMIT 2	2 row(s) returned
19	11:53:03	CREATE TABLE all_users AS SELECT id, name, surname, phone, email, birth_date, count...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
20	11:53:25	ALTER TABLE all_users MODIFY COLUMN id VARCHAR(100) NOT NULL, ADD CONSTRAI...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
21	11:53:31	ALTER TABLE products MODIFY COLUMN id VARCHAR(100) NOT NULL, ADD CONSTRAI...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
22	11:53:34	ALTER TABLE companies MODIFY COLUMN company_id VARCHAR(100) NOT NULL, AD...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
23	11:53:43	ALTER TABLE credit_cards MODIFY COLUMN id VARCHAR(100) NOT NULL, MODIFY CO...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Given that all information of tables "*american_users*" and "*european_users*" are now together in table "*all_users*", we drop these two first tables to avoid redundancy.

Fig. 4 ER STAR Diagram of the task4 database. All the relationships are 1:N to table "transactions", except for the "products" table where relationship ***has not been stablished yet.***

d. To show a list of all users with more than 80 transactions we do a subquery using the tables "*transactions*" and "*all_users*". As a result, we obtained 4 users with more than 80 transactions.



```
-- Using Subquerie
SELECT
    t.user_id AS "User Id",
    COUNT(t.id) AS "Number of Transactions"
FROM transactions t
WHERE EXISTS (
    SELECT 1
    FROM all_users au
    WHERE au.id = t.user_id
)
GROUP BY t.user_id
HAVING COUNT(t.id) > 80
ORDER BY COUNT(t.id) DESC;
```

Result Grid

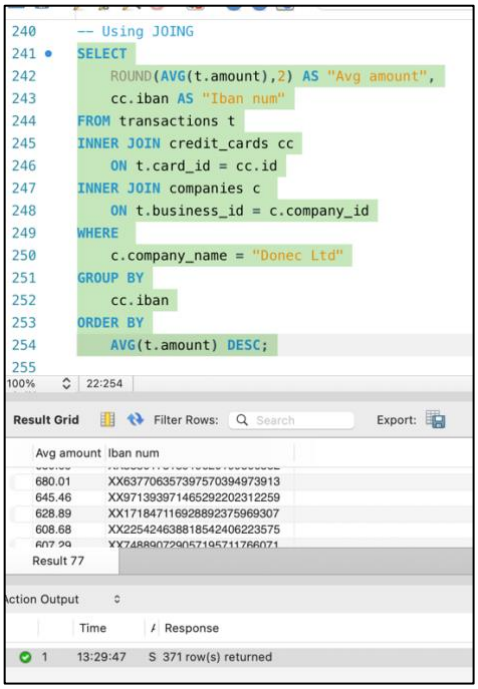
User Id	Number of Transacti...
185	110
289	94
318	91
454	81

Result 76

Time	Response
13:27:09	4 row(s) returned

Exercise 2

To show the average amount per credit card IBAN for the company 'Donec Ltd' we use the tables "*transactions*", "*credit_cards*" and "*companies*".

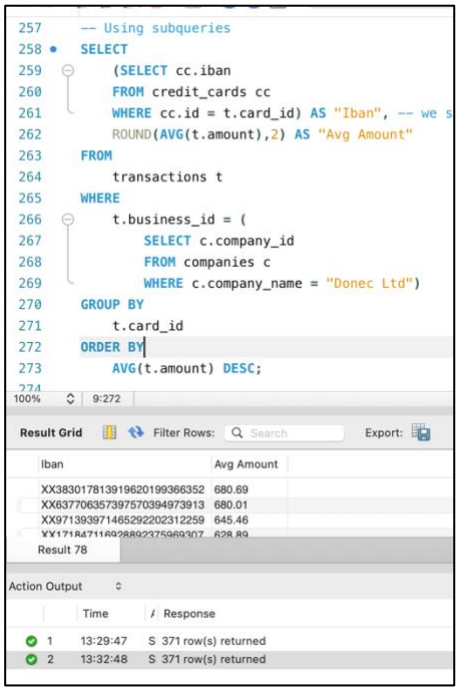


```
-- Using JOING
SELECT
    ROUND(AVG(t.amount),2) AS "Avg amount",
    cc.iban AS "Iban num"
FROM transactions t
INNER JOIN credit_cards cc
    ON t.card_id = cc.id
INNER JOIN companies c
    ON t.business_id = c.company_id
WHERE
    c.company_name = "Donec Ltd"
GROUP BY
    cc.iban
ORDER BY
    AVG(t.amount) DESC;
```

Avg amount	Iban num
680.01	XX6377063573975703949739 13
645.46	XX971393971465292202312259
628.89	XX171847116928892375969307
608.68	XX225424638818542406223575

Result 77

Time	Response
13:29:47	S 371 row(s) returned



```
-- Using subqueries
SELECT
    (SELECT cc.iban
    FROM credit_cards cc
    WHERE cc.id = t.card_id) AS "Iban", -- we s
    ROUND(AVG(t.amount),2) AS "Avg Amount"
FROM
    transactions t
WHERE
    t.business_id = (
        SELECT c.company_id
        FROM companies c
        WHERE c.company_name = "Donec Ltd")
GROUP BY
    t.card_id
ORDER BY
    AVG(t.amount) DESC;
```

Iban	Avg Amount
XX3830178139196201993663 52	680.69
XX6377063573975703949739 13	680.01
XX971393971465292202312259	645.46

Result 78

Time	Response
13:29:47	S 371 row(s) returned
13:32:48	S 371 row(s) returned

LEVEL 2

Exercise 1

To determine how many credit cards are currently active, we performed a **time-series analysis** focused on the status of the last three transactions.

We created a new table named "**active_ccards**" to classify each card as "Active" or "Inactive". A credit card is considered "Inactive" if its three most recent transactions were declined. Conversely, if at least one of those transactions was accepted, the card remains "Active".

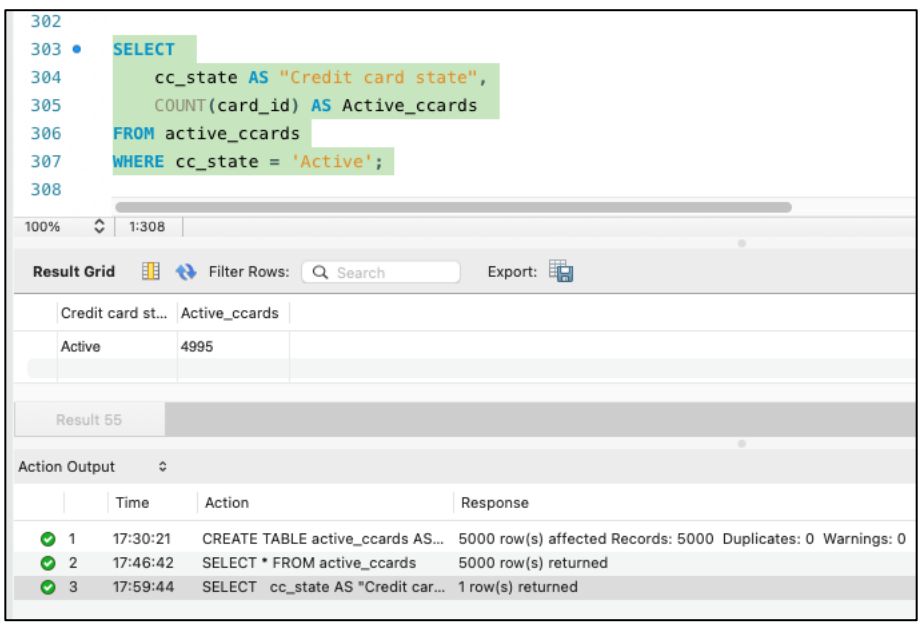
Finally, we use a new query to count the number of active credit cards (4995).

```
-- We create a new table
CREATE TABLE active_ccards AS
    WITH
        ranked_transactions AS -- First temporal table in WITH
        (SELECT card_id AS Card_id,
                declined AS Declined,
                timestamp AS Transaction_time,
            ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY timestamp DESC) AS row_num
        FROM transactions
        ),
        last3_transactions AS -- second temporal table in WITH
        (SELECT *
        FROM ranked_transactions
        WHERE row_num <= 3
        )
-- now we filter the temporary tables to create active_ccards table
    SELECT card_id,
        CASE WHEN SUM(declined) = 3 THEN 'Inactive' -- declined is a boolean arg
        ELSE 'Active'
        END AS cc_state
    FROM last3_transactions
    GROUP BY card_id;
```

Action Output

	Time	Action	Response
1	17:30:21	CREATE TABLE active_ccards AS...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

Fig. 5 New table creation to filter active credit cards



```
SELECT
    cc_state AS "Credit card state",
    COUNT(card_id) AS Active_ccards
FROM active_ccards
WHERE cc_state = 'Active';
```

100% 1:308

Result Grid Filter Rows: Search Export:

Credit card st...	Active_ccards
Active	4995

Result 55

Action Output

	Time	Action	Response
1	17:30:21	CREATE TABLE active_ccards AS...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
2	17:46:42	SELECT * FROM active_ccards	5000 row(s) returned
3	17:59:44	SELECT cc_state AS "Credit car...	1 row(s) returned

Fig. 6 Number of active credit cards based on the last three transactions

LEVEL 3

Exercise 1

The original "***transactions***" table store multiple product IDs in a single column (*product_ids*), separated by commas (e.g., "1, 5, 12"). Because a single cell contained a list of values, it was impossible to create a direct Foreign Key relationship with the "***products***" table or to perform efficient counts per product.
To solve this issue, we create an intermediate table where we use the following approach:

```
-- Create a new table to store the relationship between transactions and products
CREATE TABLE transaction_products AS
SELECT
    t.id AS transaction_id,
    jt.product_id AS Product_id
FROM transactions t
CROSS JOIN JSON_TABLE( -- CROSS JOIN: for each row in transactions generates as many row
    -- JSON TABLE generates an array that is treated as a temporary table
    CONCAT('[', t.product_ids, ']'),
    '$[*]' COLUMNS ( -- $[*] moves through every element in the array
        product_id VARCHAR(100) PATH '$'
    )
) AS jt;

DESCRIBE transaction_products;
```

Result Grid

Field	Type	Null	Key	Default	Extra
transaction_id	varchar(100)	NO		NULL	
Product_id	varchar(100)	YES		NULL	

Result 60

Action Output

	Time	A...	Response
1	18:47:58	C...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0
2	18:48:36	D...	10 row(s) returned
3	18:48:48	D...	2 row(s) returned

a. We used CONCAT function to transform the plain text string into a valid JSON Array format (e.g., [1, 5, 12]).

b. Using the JSON_TABLE function combined with a CROSS JOIN, we "exploded" each array. This means that a single transaction row with 3 products was transformed into 3 individual rows, each with one *transaction_id* and one *product_id.*

```
ALTER TABLE transaction_products
ADD CONSTRAINT pk_trans_prod_id PRIMARY KEY (transaction_id, product_id),
ADD CONSTRAINT fk_tp_to_transactions FOREIGN KEY (transaction_id) REFERENCES transactions(id),
ADD CONSTRAINT fk_tp_to_products FOREIGN KEY (product_id) REFERENCES products(id);

DESCRIBE transaction_products;

-- we count the number that each product has been sold and transaction not declined
SELECT
    p.id AS product_id,
    p.product_name,
    COUNT(tp.transaction_id) AS times_sold
FROM
    products p
LEFT JOIN
    transaction_products tp ON p.id = tp.product_id
LEFT JOIN
    transactions t ON tp.transaction_id = t.id AND t.declined = 0
GROUP BY
    p.id, p.product_name
ORDER BY
    times_sold DESC;
```

Result Grid

product_id	product_name	times_sold
52	riverlands the duel	2654
29	Tully maester Tarly	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2601
87	sith Jade	2598
48	rock Renly in	2597
33	duel warden	2597
23	riverlands north	2593
68	Stark Karstark	2589
88	Stannis warden s...	2587
28	chewbacca must...	2584
73	Dorne bastard	2584
4	warden south duel	2584
31	Lannister	2580
34	of north	2580

Result 87

Action Output

	Time	Action	Response
5	17:17:24	CREATE TABLE transaction_products AS SELECT t.id AS transaction_id, jt.product_id AS P...	253391 row(s) affected Record
6	17:17:30	ALTER TABLE transaction_products ADD CONSTRAINT pk_trans_prod_id PRIMARY KEY (transac...	253391 row(s) affected Record
7	17:17:41	SELECT p.id AS product_id, p.product_name, COUNT(tp.transaction_id) AS times_sold...	100 row(s) returned

c. Once the data is normalized (one ID per row), we establish the primary and foreign keys.

Figure 6 shows the final ER, where most tables maintain a **1:N relationship** with ***"transactio"***. To handle products, we implemented a **junction table** (***"transaction_products"***) to resolve the **N:M (Many-to-Many)** relationship between transactions and products. This allows one transaction to contain multiple products, while the same product can appear in multiple transactions.

d. Finally, we use a query to count the times that each product has been sold when transactions has not been declined.

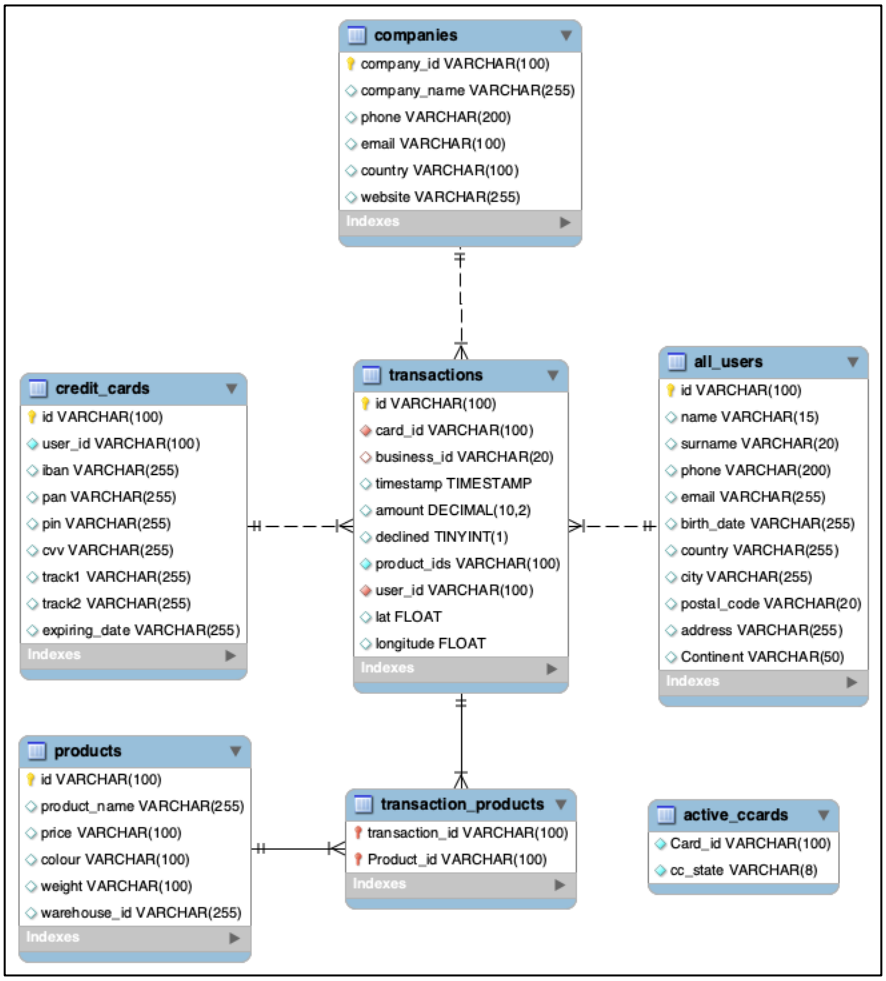


Fig. 7 Final ER that shows the relationship between all tables in the data base